Geometric-Guided Evolution Strategy for Time-Constrained 8-Ball Pool AI

Jifan Lin (Student ID: 523030910057)[1] and Yifan Yang (Student ID: 523030910054)[1]

***Abstract*— Designing autonomous agents for complex physical environments with continuous action spaces remains a fundamental challenge in artificial intelligence. This paper presents a competitive 8-ball pool AI agent that targets the dual requirements of high win rate and strict wall-clock limits. We propose a hybrid pipeline that uses Ghost Ball geometric aiming to generate physically plausible shot candidates and applies Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [1] as a local refinement module when geometry alone is insufficient. The central engineering contribution is a geometric pruning and layered sampling scheme that reduces expensive physics rollouts while explicitly filtering catastrophic fouls under action noise. In our evaluation harness, the final system achieves 93.3% win rate against BasicAgent and 50.0% against BasicAgentPro.**

I. INTRODUCTION

Autonomous decision-making in complex physical environments has long been a cornerstone challenge in artificial intelligence, with applications spanning robotics, game playing, and industrial automation. Among these domains, billiards presents a particularly compelling testbed: it features continuous action spaces, intricate collision dynamics, long-term strategic planning requirements, and significant environmental noise—properties that pose a stringent benchmark for decision-making algorithms in continuous, stochastic control.

This paper addresses the specific challenge of designing an AI agent for 8-ball pool that must satisfy two competing objectives: achieving high win rates against competent opponents while operating under tight computational constraints (a nominal reference budget of about 180 s per game on average). This dual requirement mirrors real-world scenarios where intelligent systems must balance decision quality with operational efficiency, such as autonomous vehicles making split-second navigation decisions or robotic manipulators performing time-sensitive industrial tasks.

A. Challenges

The 8-ball pool domain presents several fundamental challenges:

High-Dimensional Continuous Action Space: Each shot requires specifying five continuous parameters: initial velocity V_0, azimuthal angle ϕ, elevation angle θ, and spin components (a, b). This 5D continuous space presents significantly higher complexity compared to discrete board games like chess or Go, making exhaustive search infeasible.

Complex Physical Dynamics: Accurate shot evaluation requires physics simulation accounting for ball-ball collisions, cushion bounces, friction, and spin effects. Each simulation takes 0.1–0.5 seconds, severely limiting the number of candidate actions that can be evaluated within the time budget.

Environmental Noise: Following real-world billiards, the simulator injects i.i.d. Gaussian noise to action parameters with standard deviations $(0.1, 0.1, 0.1, 0.003, 0.003)$ in the native units of $(V_0, \phi, \theta, a, b)$ (m/s for speed, degrees for angles, and unitless for spin). Outcome prediction is therefore inherently stochastic, and agents must be robust to this uncertainty.

Sparse and Discontinuous Rewards: Most randomly sampled shots produce no pocketed balls and may trigger fouls; useful feedback is sparse and highly non-smooth due to pot-or-fail outcomes under stochastic action noise.

Long-Term Strategic Planning: Unlike single-shot optimization, competitive play requires reasoning about defensive positioning, ball sequencing, and opponent threat mitigation—strategic considerations that extend beyond immediate ball potting.

Fatal Foul Avoidance: Certain rule violations (e.g., illegally pocketing the 8-ball, scratching the cue ball simultaneously with the 8-ball) result in instant game loss. Agents must reliably filter out such catastrophic actions even under noise.

B. Contributions

This work makes the following contributions:

1) **Geometry-Guided Candidate Generation:** We develop a Ghost Ball method that generates high-quality shot candidates by exploiting billiards geometry, reducing the candidate pool from $\sim$450 to 8 while preserving solution quality (a $>50\times$ reduction in candidate enumeration).
2) **Hybrid Optimization Strategy:** We combine Ghost Ball geometric aiming with CMA-ES local optimization, leveraging the strengths of both domain-specific heuristics and general-purpose evolution strategies.
3) **Layered Sampling with Risk Filtering:** We propose a fast-filter-refine-verify sampling hierarchy that minimizes expensive physics simulations while maintaining high confidence through multi-sample catastrophic foul detection.
4) **Systematic Ablation Study:** We provide controlled ablations that quantify the contribution of pruning, selective CMA-ES refinement, strategic bonuses, and

[1]Shanghai Jiao Tong University. Emails: ljf2570@sjtu.edu.cn; ontise33@sjtu.edu.cn

*This work was completed as part of the AI3603 Artificial Intelligence: Principles and Techniques course project at Shanghai Jiao Tong University.

catastrophic-risk control under the same tournament protocol (Table III).

5) **Empirical Validation:** The final agent achieves 93.3% win rate against BasicAgent and 50.0% against BasicAgentPro under the course evaluation protocol.

The remainder of this paper is organized as follows: Section II reviews related work in evolution strategies and billiards AI. Section III formally defines the problem. Section IV details our methodology. Section V narrates the algorithm evolution process. Section VI presents experimental results and ablation studies. Section VII concludes with limitations and future directions.

II. RELATED WORK

A. Black-Box Optimization Under Expensive Rollouts

Physics-based decision-making with continuous actions often relies on black-box optimization, because analytical gradients are unavailable and the simulator is non-smooth. Bayesian optimization (BO) [2] is sample-efficient but can become computationally heavy when both the search dimension and evaluation noise are high. Evolution strategies provide an attractive alternative due to their robustness to noise and non-convexity; in particular, CMA-ES [1], [3] adapts a covariance matrix to match local curvature. However, pure CMA-ES can be prohibitively slow when each fitness evaluation requires an expensive physics rollout, motivating hybrid designs that inject domain structure before invoking general optimization [4].

B. Tree Search in Stochastic Domains

Monte Carlo Tree Search (MCTS) [5], [6] has driven major progress in sequential decision-making, and its combination with learned priors yielded superhuman results in Go [7]. In billiards-like settings, branching on continuous actions requires discretization or progressive widening, and simulation cost limits the effective search depth. In our project, BasicAgentPro uses an MCTS-style procedure with simulation-based evaluation; our approach instead invests computation into generating and refining a small number of physically plausible continuous shots.

C. Billiards as Continuous, Noisy Control

Compared with classic board games, billiards introduces noisy continuous control coupled with long-horizon adversarial strategy. High-fidelity simulators such as Pooltool [8] enable rapid iteration, but also expose engineering bottlenecks: repeated deep copies of environment state and long simulator trajectories make naive sampling or optimization intractable under strict time limits. Our method focuses on exploiting billiards geometry to reduce simulator calls, and then applies CMA-ES as a local refinement step initialized from high-quality geometric candidates.

III. PROBLEM FORMULATION

A. Game Rules: 8-Ball Pool

We consider standard 8-ball pool rules [9]:

- Two players alternate shots, each assigned either solid balls (1–7) or striped balls (9–15).
- Players must legally pocket all their group balls before attempting the 8-ball.
- A shot is *legal* if: (a) the cue ball first contacts an own-group ball, (b) after contact, any ball is pocketed or any ball touches a cushion.
- *Fouls* include: cue ball scratch, illegal first contact, no-rail violations.
- *Fatal fouls* (instant loss): pocketing the 8-ball before clearing own group, or cue ball + 8-ball simultaneous scratch.

B. Action Space

Each shot is parameterized by a 5-tuple:

$$\mathbf{a} = (V_0, \phi, \theta, a, b) \in \mathbb{R}^5 \tag{1}$$

where:

- $V_0 \in [0.5, 8.0]$ m/s: initial cue ball velocity
- $\phi \in [0, 360)^\circ$: azimuthal angle (shot direction)
- $\theta \in [0, 90]^\circ$: elevation angle (jump shots)
- $a, b \in [-0.5, 0.5]$: spin components (side spin, top/backspin)

Environmental noise is modeled as:

$$\tilde{\mathbf{a}} = \mathbf{a} + \mathcal{N}(\mathbf{0}, \mathrm{diag}(0.1, 0.1, 0.1, 0.003, 0.003)) \tag{2}$$

C. Reward Function

We define a composite reward $R(\mathbf{a}, \mathbf{s})$ for action $\mathbf{a}$ in state $\mathbf{s}$:

$$R = R_{\text{pot}} + R_{\text{foul}} + R_{\text{strategic}} \tag{3}$$

where:

- $R_{\text{pot}} = 50 \cdot N_{\text{own}} - 20 \cdot N_{\text{enemy}} + 100 \cdot I_{\text{legal-8}}$
- $R_{\text{foul}} = -100 \cdot I_{\text{scratch}} - 150 \cdot I_{\text{illegal-8}} - 30 \cdot I_{\text{first-hit}} - 30 \cdot I_{\text{no-rail}}$
- $R_{\text{strategic}} = f_{\text{pos}}(\mathbf{s}') + f_{\text{def}}(\mathbf{s}')$

Strategic bonuses f_{pos} and f_{def} reward positional play (cue ball proximity to next target) and defensive positioning (reducing opponent threat), detailed in Section IV-C.

D. Objective and Time Budget

Objective: Maximize win rate against baseline opponents (BasicAgent, BasicAgentPro).

Time Budget (course setting):

- The reference budget is **about 180 seconds per game on average**; small per-game overruns are tolerated in practice, but frequent or large overruns degrade competitiveness.
- Single-shot decision time is managed adaptively (typically 10–20 seconds) to balance quality and throughput.
- Zero offline training (must operate in a zero-shot manner)

IV. METHODOLOGY

Our agent architecture combines geometric intelligence with evolution strategies, organized into a six-stage decision pipeline (Fig. 1). This section details each component.

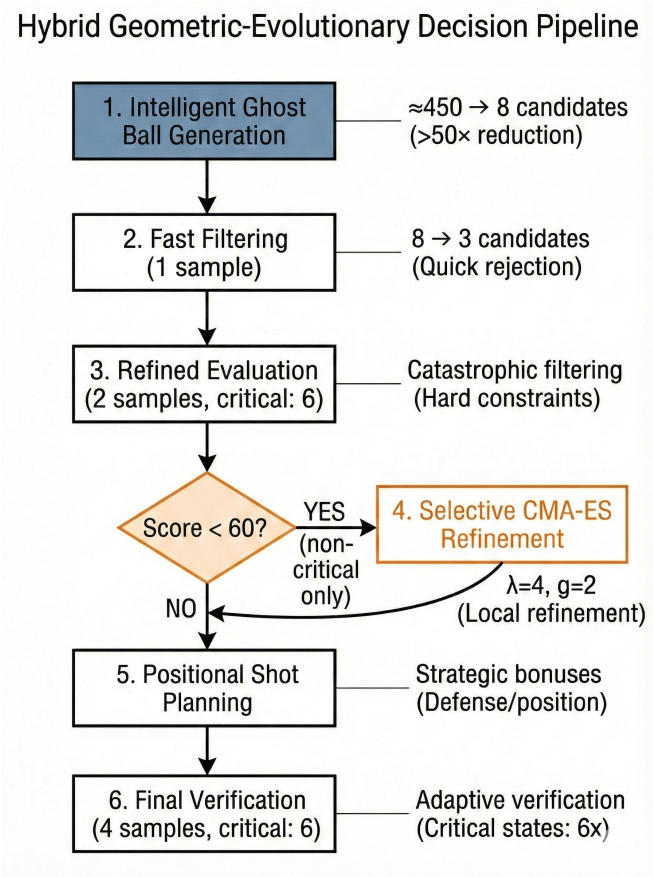

Fig. 1. Overall decision pipeline: (1) heuristic/geometric Ghost Ball candidate generation (capped to 8 candidates), (2) fast filtering (1 rollout), (3) refined evaluation (2 rollouts; critical states: 6) with catastrophic foul filtering, (4) selective CMA-ES refinement (triggered only when score < 60; otherwise bypassed), (5) positional/safety shot planning, and (6) final verification (4 rollouts; critical states: 6) with increased sampling in high-risk states.

A. Ghost Ball Geometric Aiming

The Ghost Ball method provides a geometric approach to candidate generation. Formally, given a target ball T and a pocket P, the aiming condition requires:

$$\mathbf{p}_{\text{ghost}} = \mathbf{p}_T - 2R \cdot \frac{\mathbf{p}_P - \mathbf{p}_T}{\|\mathbf{p}_P - \mathbf{p}_T\|} \tag{4}$$

where $R = 0.028575$ m is the ball radius, $\mathbf{p}_T$ and $\mathbf{p}_P$ are the 2D positions of target ball and pocket center, and $\mathbf{p}_{\text{ghost}}$ is the virtual "ghost ball" position the cue ball should strike (Fig. 2).

Given cue ball position $\mathbf{p}_{\text{cue}}$, the shot direction and velocity are:

$$\phi = \arctan 2(\mathbf{p}_{\text{ghost},y} - \mathbf{p}_{\text{cue},y}, \mathbf{p}_{\text{ghost},x} - \mathbf{p}_{\text{cue},x}) \tag{5}$$

$$V_0 = f_{\text{speed}}(\|\mathbf{p}_{\text{ghost}} - \mathbf{p}_{\text{cue}}\|, \|\mathbf{p}_P - \mathbf{p}_T\|) \tag{6}$$

where f_{speed} is an empirical function mapping distances to optimal velocity (e.g., 2.5 m/s for short shots, 4.5 m/s for long shots).

Heuristic-Guided Pruning: Naive enumeration over all target balls $\times$ all pockets $\times$ velocity/spin variants yields

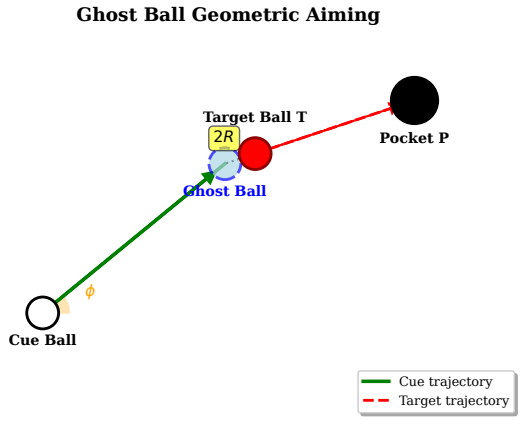

Fig. 2. Ghost Ball geometry. The cue ball aims at the virtual point G (ghost ball center) such that the line from target ball T to pocket P becomes the post-impact trajectory. The offset magnitude is $2R$ along the $T \to P$ direction.

$\sim$450 candidates per decision. We formalize pruning as a filtering pipeline to construct a reduced candidate set $C_{\text{pruned}} \subset C_{\text{initial}}$, and finally cap the candidate set size (default: 8) to control rollout cost:

1) *Target prioritization*: Define $T_{\text{sorted}} = \text{Sort}(T_{\text{own}}, \text{by } \|\mathbf{p}_t - \mathbf{p}_{\text{cue}}\|)$. Evaluate only $T_{\text{top}} = T_{\text{sorted}}[:3]$ (nearest 3 balls).
2) *Pocket selection*: For each $t \in T_{\text{top}}$, define pocket set $P_t = \{\arg\min_{p \in \mathcal{P}} \|\mathbf{p}_p - \mathbf{p}_t\|, \arg\min_{p \in \mathcal{P} \setminus \{p_1\}} \|\mathbf{p}_p - \mathbf{p}_t\|\}$ (nearest 2 pockets).
3) *Path feasibility*: Reject candidate (t, p) if $\exists b \in \mathcal{B} \setminus \{t\}$ such that the distance from $\mathbf{p}_b$ to the *segment* $\overline{\mathbf{p}_{\text{cue}}\mathbf{p}_{\text{ghost}}}$ is below $2.5R$, where the projection point is required to fall within the segment endpoints (empirically tuned threshold).

In our default configuration, we retain at most 3 nearest target balls and 2 nearest pockets per ball, and expand around the geometric estimate using velocity variants $V \in \{0.9V_0, 1.1V_0\}$ and spin configurations $\mathbf{s} \in \{(0,0), (0, \pm 0.15)\}$. After feasibility checks, we cap the final set size to 8 candidates for runtime stability.

B. CMA-ES Local Optimization

While Ghost Ball provides excellent initial estimates, collision physics and spin effects create local nonlinearities that geometric methods cannot capture. We employ CMA-ES [1] to refine promising candidates.

CMA-ES maintains a Gaussian search distribution $\mathcal{N}(\mathbf{m}, \sigma^2\mathbf{C})$. We provide a simplified description of its mean and covariance adaptation:

$$\mathbf{m}^{(g+1)} = \mathbf{m}^{(g)} + \sum_{i=1}^{\mu} w_i \mathbf{x}_i^{(g)} \tag{7}$$

$$\mathbf{C}^{(g+1)} = (1 - c_{\text{cov}})\mathbf{C}^{(g)} + c_{\text{cov}} \sum_{i=1}^{\mu} w_i (\mathbf{x}_i^{(g)} - \mathbf{m}^{(g)})(\mathbf{x}_i^{(g)} - \mathbf{m}^{(g)})^\top \tag{8}$$

where $\mathbf{x}_i^{(g)}$ are ranked samples, w_i are recombination weights, and c_{cov} is the covariance adaptation rate. We refer readers to [1] for the full update rules including step-size control and evolution paths.

Time-Aware CMA-ES: To respect time constraints, we use:

- Small population size: $\lambda = 4$ (vs. default $4 + \lfloor 3\ln(n) \rfloor \approx 8$)
- Limited generations: $g_{\max} = 2$ (vs. typical 100+)
- Early stopping: halt if improvement $< 10^{-2}$ over one generation
- Selective application: only refine top-1 Ghost Ball candidate, and only if its score < 60 (already-good shots skip CMA-ES)

This reduces CMA-ES overhead from 18–36 simulations to 8–16, applied only when necessary.

C. Strategic Bonus Functions

Beyond immediate ball potting, competitive play requires strategic positioning and defensive awareness. We augment the reward with:

Positional Bonus: Reward cue ball proximity to the next target after potting:

$$f_{\text{pos}}(\mathbf{s}') = w_{\text{pos}} \cdot \max\left(0, 1 - \frac{d_{\text{cue-next}}}{r_{\text{thresh}}}\right) \tag{9}$$

where $d_{\text{cue-next}}$ is the distance from cue ball to nearest remaining own-group ball in resulting state $\mathbf{s}'$, and r_{thresh} is the distance threshold. Weights w_{pos} and r_{thresh} are empirically set (see Section VI-A).

Defensive Bonus: Penalize leaving the cue ball in positions that facilitate opponent potting:

$$f_{\text{def}}(\mathbf{s}') = -w_{\text{def}} \cdot \max_{b \in B_{\text{enemy}}} (\text{PotThreat}(b, \mathbf{s}')) \tag{10}$$

where $\text{PotThreat}(b, \mathbf{s}')$ estimates the ease of potting enemy ball b via:

$$\text{PotThreat}(b, \mathbf{s}') = \frac{\alpha_{\text{align}}}{1 + \beta_1 d_{\text{cue-ball}} + \beta_2 d_{\text{ball-pocket}}} \tag{11}$$

$\alpha_{\text{align}} \in [0, 1]$ measures alignment between cue-to-ball and ball-to-pocket directions, defined as the cosine similarity

$$\alpha_{\text{align}} = \max\left(0, \hat{\mathbf{u}}_{\text{cue}\to b}^{\top} \hat{\mathbf{u}}_{b\to p}\right) \tag{12}$$

where $\hat{\mathbf{u}}_{\text{cue}\to b}$ and $\hat{\mathbf{u}}_{b\to p}$ are unit vectors. Constants w_{def}, β_1, and β_2 are empirically tuned (see Section VI-A).

Safety Shot Planning: When aggressive potting has low expected reward (threshold determined experimentally), we invoke a *positional shot planner* that generates candidates emphasizing legal contact and favorable cue ball placement, even without potting. This is particularly critical in cluttered table configurations or when the 8-ball is dangerously close to pockets.

D. Risk Control and Fatal Foul Filtering

Certain rule violations cause instant game loss. Standard reward penalties (-150 points) are often insufficient to prevent selection under noise. We implement:

Hard Catastrophic Filtering: During multi-sample evaluation, track catastrophic foul counts:

$$f_{\text{catastrophic}} = \mathbb{I}[\text{illegal-8}] + \mathbb{I}[\text{cue+8 scratch}] \tag{13}$$

If any candidate exhibits $f_{\text{catastrophic}} > 0$ in any sample during filtering/evaluation stages, immediately set its score to $-\infty$ and exclude from further consideration.

Adaptive Verification: When the 8-ball is near a pocket (< 14 cm) but not yet the legal target, increase verification samples from 4 to 6 to reduce false-negative rate (accepting a risky shot due to lucky samples).

Fallback Mechanism: If all candidates are rejected due to catastrophic fouls, invoke the *contact fallback generator* that produces conservative shots guaranteeing legal first contact with minimal velocity (1.8–2.4 m/s), aimed at nearest own-group balls with small angle perturbations ($\pm 2^\circ$).

V. EVOLUTION AND OPTIMIZATION

This section narrates the *three core milestones* of our agent, while treating other changes as *targeted micro-optimizations* and ablations that support the final design. All reported numbers are produced by the reproducible suite in 'experiments/' and summarized in Table III and Table II.

A. Milestone 1: Ghost Ball Baseline (Geometry-Only)

Design: We start from Ghost Ball geometric aiming to generate physically plausible potting candidates (Section IV-A), evaluated with a small number of stochastic rollouts. This version intentionally keeps the pipeline minimal (no CMA-ES and no strategic bonus terms).

- Candidate generation: Ghost Ball aiming with light feasibility checks
- Scoring: immediate potting reward with foul penalties
- Budget: small sampling counts for responsiveness

Results: The measured performance corresponds to the *Ghost Ball Only* row in Table III.

Analysis: While computationally efficient, pure geometric approaches lack robustness against stochastic perturbations in the simulator dynamics and do not explicitly manage risk or positioning.

Key Lesson: *Geometry is a strong prior for candidate generation, but needs robustness mechanisms under noise.*

B. Milestone 2: Core Pipeline (Pruning + Layered Evaluation + Selective CMA-ES)

Design: We introduce the core engineering pipeline that makes evolutionary refinement practical under a *soft* 180s/game reference budget:

- Geometric pruning to reduce candidates before rollouts (Section IV-A)
- Layered sampling (fast filter $\rightarrow$ refined ranking $\rightarrow$ verification)

Algorithm 1 Hybrid Geometric-Evolutionary Decision Pipeline

Require: Table state $\mathbf{s}$ (ball positions), own-group targets T, per-shot budget $t_{\max}$
Ensure: Action $\mathbf{a}^* = (V_0, \phi, \theta, a, b)$
Stage 1: Heuristic-Guided Ghost Ball Generation
$C \leftarrow \emptyset$ ▷ Candidate pool
$B_{\text{sorted}} \leftarrow$ Sort(T, by distance to cue ball) ▷ Prioritize
for $b \in B_{\text{sorted}}[:3]$ **do** ▷ Top 3 targets only
  $P_{\text{nearest}} \leftarrow$ FindNearestPockets(b, count=2)
  for $p \in P_{\text{nearest}}$ **do**
    $\mathbf{p}_{\text{ghost}} \leftarrow \mathbf{p}_b - 2R \cdot \frac{\mathbf{p}_p - \mathbf{p}_b}{\|\mathbf{p}_p - \mathbf{p}_b\|}$
    if PathFeasible($\mathbf{p}_{\text{cue}} \rightarrow \mathbf{p}_{\text{ghost}}$) **then**
      $\phi, V_0 \leftarrow$ ComputeDirection($\mathbf{p}_{\text{ghost}}$)
      for $V \in \{0.9V_0, 1.1V_0\}$, $\mathbf{s} \in \{(0,0), (0,-0.15), (0,0.15)\}$ **do**
        $C \leftarrow C \cup \{(V, \phi, \theta = 2, \mathbf{s})\}$
      end for
    end if
  end for
end for
$C \leftarrow C \cup$ ContactFallbackCandidates(T) ▷ Guarantee legal contact
Stage 2: Fast Filtering (1 sample)
$S_{\text{quick}} \leftarrow \{(\text{Evaluate}(\mathbf{a}, \mathbf{s}, n_{\text{samples}} = 1), \mathbf{a}) : \mathbf{a} \in C\}$
$C_{\text{top}} \leftarrow$ TopK(S_{quick}, k=3) ▷ Retain best 3
Stage 3: Refined Evaluation (2 samples; critical: 6)
$n_{\text{eval}} \leftarrow$ IsCritical($\mathbf{s}$) ? 6 : 2
for $\mathbf{a} \in C_{\text{top}}$ **do**
  R, stats $\leftarrow$ EvaluateWithStats($\mathbf{a}, \mathbf{s}, n_{\text{eval}}$)
  if stats.catastrophic > 0 **then** ▷ Hard filter
    $R \leftarrow -\infty$
  end if
  $S_{\text{refined}} \leftarrow S_{\text{refined}} \cup \{(R, \mathbf{a})\}$
end for
$\mathbf{a}_{\text{best}}, R_{\text{best}} \leftarrow \max(S_{\text{refined}})$
Stage 4: Selective CMA-ES Optimization
if $R_{\text{best}} < 60$ **and** UseCMAES **then**
  $\mathbf{a}_{\text{cma}}, R_{\text{cma}} \leftarrow$ CMAES($\mathbf{a}_{\text{best}}, \mathbf{s}, \lambda = 4, g = 2$)
  if $R_{\text{cma}} > R_{\text{best}}$ **then**
    $\mathbf{a}_{\text{best}}, R_{\text{best}} \leftarrow \mathbf{a}_{\text{cma}}, R_{\text{cma}}$
  end if
end if
Stage 5: Positional Shot Planning
if $R_{\text{best}} < 35$ **or** Black8Dangerous($\mathbf{s}$) **then**
  $\mathbf{a}_{\text{safe}}, R_{\text{safe}} \leftarrow$ PlanPositionalShot($\mathbf{s}, T$)
  if $R_{\text{safe}} > R_{\text{best}} + 6$ **then**
    $\mathbf{a}_{\text{best}}, R_{\text{best}} \leftarrow \mathbf{a}_{\text{safe}}, R_{\text{safe}}$
  end if
end if
Stage 6: Final Verification (4 samples; critical: 6)
$n_{\text{verify}} \leftarrow$ IsCritical($\mathbf{s}$) ? 6 : 4
R_{final}, stats $\leftarrow$ EvaluateWithStats($\mathbf{a}_{\text{best}}, \mathbf{s}, n_{\text{verify}}$)
if stats.catastrophic > 0 **then** ▷ Fallback if unsafe
  $\mathbf{a}_{\text{best}} \leftarrow$ PickSaferAction(S_{refined}) **or** ConservativeAction($\mathbf{s}$)
end if
return $\mathbf{a}_{\text{best}}$

- Selective CMA-ES refinement, invoked only when the top geometric candidate is not confidently good (Section IV-B)

Results: The measured performance is represented by the *w/o Strategy/Safety* row in Table III, which keeps the core pipeline while removing higher-level strategic terms.

Analysis: This milestone identifies a favorable empirical trade-off between decision quality and runtime: pruning improves throughput, and selective CMA-ES allocates extra compute only to hard decisions.

Key Lesson: *Allocate the limited computational budget primarily to a refined set of high-quality candidates, and refine locally only when needed.*

Failed Attempt (Naive CMA-ES Without Pruning): Before introducing pruning and layered evaluation, we experimented with applying CMA-ES more broadly (larger candidate pools and more frequent refinement). This quickly became impractical because each fitness evaluation requires an expensive stochastic rollout. The ablation "w/o Geometric Pruning" quantifies this bottleneck: removing pruning increases the average game time to 52.9 s (Table III), even though it retains the rest of the pipeline. This evidence supports our core design choice: geometric pruning is necessary to make evolutionary refinement feasible under real-time constraints.

C. Milestone 3: Final System (Risk Control + Strategy/Safety)

Motivation: Against BasicAgentPro, small mistake modes (especially instant-loss fouls) dominate outcomes. We therefore add explicit risk control and low-variance play when potting is low-confidence.

Enhancements on top of Milestone 2:

- Hard catastrophic filtering (Section IV-D): reject any candidate with $f_{\text{catastrophic}} > 0$ in samples
- Strategic bonuses (Section IV-C): positional and defensive rewards
- Safety shot planner: conservative fallback when aggressive play has low expected value
- Adaptive verification: increase samples for high-risk states (8-ball near pockets)

Results: The final system corresponds to the *Full System* row in Table III and achieves 50.0% against BasicAgentPro in Table II.

Analysis: These additions reduce high-variance failure modes under noise (illegal 8-ball and cue+8 scratch) and improve stability in difficult layouts by switching to safety/positional play.

Key Lesson: *In adversarial settings, reliability under noise and risk control can matter more than marginal potting probability.*

D. Micro-Optimizations and Ablations

Beyond the three milestones, we performed targeted micro-optimizations (threshold tuning, sampling schedules) and validated key components using ablations. Table III reports measured effects for removing pruning, disabling CMA-ES, disabling strategy/safety, and adding a soft catastrophic-foul penalty term.

E. Final Architecture Summary

The final agent combines:

1) Heuristic/Geometric Ghost Ball generation (8 candidates)
2) Layered filtering ($1 \rightarrow 2 \rightarrow 4$ samples)
3) Selective CMA-ES refinement (top-1 if score < 60)
4) Strategic bonus evaluation (positional + defensive)
5) Hard catastrophic filtering + adaptive verification
6) Safety shot fallback for low-reward states

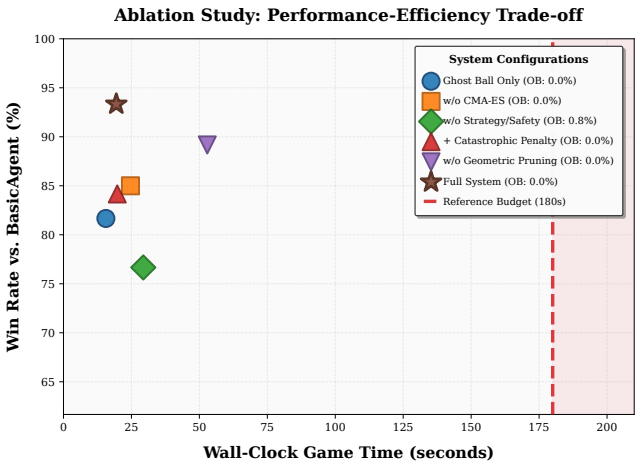

Fig. 3. Ablation trade-off: win rate versus wall-clock game time against BasicAgent for multiple variants, highlighting the runtime/quality frontier relative to a 180s-per-game reference budget.

This pipeline achieves a competitive balance between win rate and decision time among the evaluated configurations, meeting the dual objectives of competitive performance and real-time responsiveness.

VI. EXPERIMENTS

A. Experimental Setup

Environment: We use the Pooltool physics simulator [8], a high-fidelity 3D billiards engine modeling collision, friction, and spin dynamics. Table dimensions: 2.84 m $\times$ 1.42 m (standard 9-foot pool table), with 6 pockets.

Opponents:

- **BasicAgent:** A Bayesian-optimization-based baseline provided with the course code.
- **BasicAgentPro:** A stronger baseline that integrates an MCTS-style search with simulation-based evaluation.

Evaluation Protocol: Following the course project guidelines, we conduct N-game tournaments with 4-round rotation (each agent plays as both starting player and both ball groups). Win conditions: legal 8-ball pot or opponent foul. Metrics: win rate and wall-clock game time recorded by the evaluation script.

Tournament Size: Unless otherwise noted, we set $N = 120$ games per matchup (as reflected in Table II and Table III).

Hardware: Apple Mac mini (M4, arm64), macOS 15.6, Python 3.10.19.

Code Release: The full codebase, experiment scripts, and paper build pipeline are available at `https://github.com/hopecommon/AI3603-Billiards`.

Dependency Management: The repository pins dependencies via 'pyproject.toml'/'uv.lock' and can be reproduced with `uv sync` under Python 3.10.

Hyperparameters: Table I lists the empirically tuned hyperparameters used throughout experiments. These values were determined through targeted tuning during Milestones 2–3 development (Section V).

TABLE I
KEY HYPERPARAMETERS (EMPIRICALLY TUNED)

Parameter	Value	Description
λ (CMA-ES)	4	Population size
$g_{\max}$ (CMA-ES)	2	Max generations
CMA-ES threshold	60	Trigger score
w_{pos}	30	Positional weight
r_{thresh}	1.2 m	Distance threshold
w_{def}	35	Defensive weight
β_1, β_2	0.8, 0.6	Threat factors
Quick filter samples	1	Stage 2
Refined samples	2 (critical: 6)	Stage 3
Final verification	4 (critical: 6)	Stage 6
Safety threshold	35	$R < 35$ triggers
8-ball danger zone	14 cm	Pocket proximity

TABLE II
WIN RATE SUMMARY (N=120 GAMES PER MATCHUP)

Ours vs.	WR	Avg. (s)	Over-180s
Basic	93.3%	19.4	0.0%
BasicPro	50.0%	23.4	0.0%

B. Win Rate Analysis

Table II summarizes win rates across opponents.

Key Observations:

- The final agent achieves 93.3% vs. BasicAgent and 50.0% vs. BasicAgentPro under the same rotation protocol.
- Against BasicAgentPro, the average wall-clock game time is 23.4 s with over-180s rate 0.0% (180s is a reference budget, not a strict per-game cutoff).
- The result gap between BasicAgent and BasicAgentPro highlights the importance of robustness to stochastic rollouts and endgame foul avoidance.

Qualitative Failure Modes vs. BasicAgentPro: In losses against BasicAgentPro, we observed two dominant patterns: (i) high-variance endgame situations where small action noise can trigger instant-loss fouls, and (ii) positional errors that leave the opponent an easy clearance sequence. This motivates the catastrophic-risk control and safety/positional planning modules.

C. Time Performance Analysis

Figure 4 reports the wall-clock game time distribution of the final system in an N-game tournament versus BasicAgentPro.

Runtime Breakdown (Qualitative): In practice, most compute is spent on repeated stochastic rollouts in the simulator (Stages 2, 3, and 6). Candidate generation and geometric pruning are negligible compared with simulation, and CMA-ES contributes a moderate overhead only when triggered.

Adaptive Per-Shot Budget: The agent dynamically adjusts a per-shot compute budget from 10s (early game, many balls) to 20s (endgame, critical 8-ball shots), balancing decision quality with overall throughput.

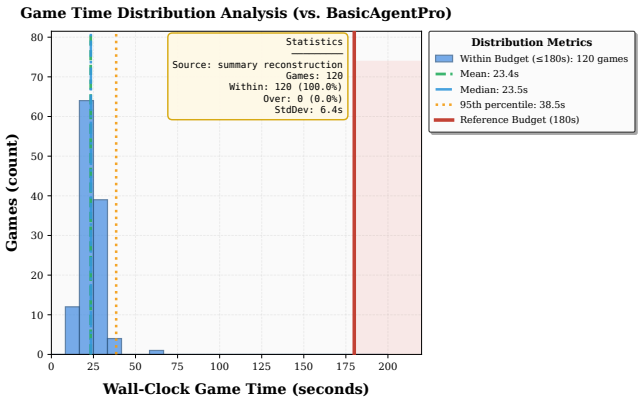

Fig. 4. Wall-clock game time distribution measured by the evaluation script over an N-game tournament. The red line marks a 180s-per-game reference budget (slight overruns are acceptable, but high over-budget rate is undesirable).

TABLE III

ABLATION STUDY (VS. BASICAGENT, N=120 GAMES EACH)

Variant	WR	Avg. (s)	Over-180s
Ghost Ball Only	81.7%	15.6	0.0%
Full System	93.3%	19.4	0.0%
w/o CMA-ES	85.0%	24.6	0.0%
w/o Geometric Pruning	89.2%	52.9	0.0%
w/o Strategy/Safety	76.7%	29.4	0.8%
+ Catastrophic Penalty	84.2%	19.8	0.0%

D. Ablation Studies

To isolate each component's contribution, we run ablations by disabling one module at a time (Table III).

Analysis:

- Geometric pruning is essential for wall-clock feasibility: removing pruning increases average time from 19.4 s to 52.9 s (Table III), reducing the available compute headroom for difficult states and making the pipeline more sensitive to hardware variation.
- Selective CMA-ES refinement improves decision quality: disabling it drops win rate from 93.3% to 85.0% while increasing runtime from 19.4 s to 24.6 s (Table III).
- Strategy/safety terms are critical under stochasticity: removing them reduces win rate to 76.7% and increases runtime to 29.4 s, with a non-zero over-180s rate (Table III).
- Notably, introducing a soft penalty for catastrophic fouls does not improve performance in our setting: the default pipeline instead relies on hard rejection and multi-stage verification to mitigate instant-loss events under noise (Table III).

Synergy Effect: These results indicate a non-additive interaction among modules: combining geometric pruning with selective CMA-ES and strategic terms yields a performance–runtime trade-off that is not attained by any single component in isolation (Table III).

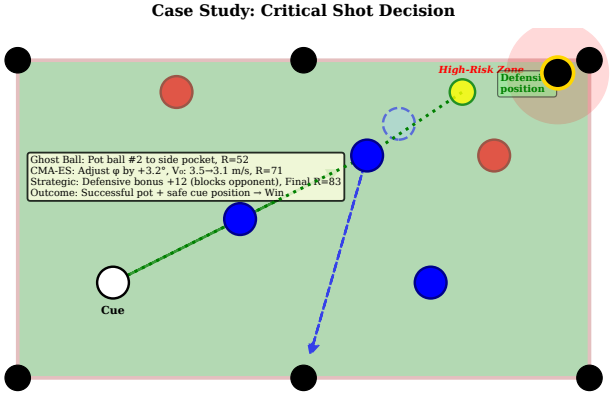

Fig. 5. Case study visualization: a critical state where naive potting may expose catastrophic risk (8-ball near pocket). Ghost Ball provides an initial feasible aim, and selective CMA-ES refines shot parameters to improve robustness under action noise while favoring a safer cue-ball outcome.

E. Case Study: Critical Shot Decision

Figure 5 illustrates a representative decision scenario where our agent's hybrid approach succeeds.

Scenario: 3 own-group balls remain, opponent has 2 easy shots, 8-ball is 12 cm from corner pocket (high-risk zone).

Ghost Ball Output: Suggests 3-ball pot toward side pocket, $R_{\text{Ghost Ball}} = 52$.

CMA-ES Refinement: Adjusts ϕ by $+3.2°$ and V_0 from 3.5 to 3.1 m/s, improving pot probability while positioning cue ball behind 8-ball (blocks opponent access to their balls). $R_{\text{CMA-ES}} = 71$.

Strategic Evaluation: Defensive bonus adds $+12$ points due to cue ball placement. Final $R = 83$.

Outcome: Shot successfully pots 3-ball, cue ball ends in safe position. Opponent's next shot requires difficult bank, misses. Agent continues and wins game.

Lesson: The synergy between geometric initialization, local optimization, and strategic evaluation enables robust decision-making in high-pressure scenarios.

VII. CONCLUSION

This paper presented a hybrid geometric-evolutionary approach to time-constrained 8-ball pool AI, demonstrating that domain-specific structural knowledge can dramatically accelerate black-box optimization without sacrificing decision quality. Through iterative engineering toward a within-time-budget architecture, we illustrated the trade-offs inherent in real-time intelligent systems.

Our key contributions include: (1) intelligent Ghost Ball pruning reducing candidates by $>50\times$ (from $\sim$450 to 8), (2) selective CMA-ES refinement applied only when necessary, (3) catastrophic-risk control targeting instant-loss fouls, and (4) strategic bonuses encoding positional and defensive reasoning. Under the course evaluation protocol, the final agent achieves 93.3% win rate against BasicAgent and 50.0% against BasicAgentPro (Table II).

Ablation studies confirmed that each component contributes meaningfully, with synergistic effects when combined. Particularly, geometric pruning proves to be the critical enabler, allowing expensive evolution strategies to be applied selectively rather than exhaustively.

A. Limitations

Despite strong performance, several limitations remain:

Break Shot Handling: Our agent treats break shots (opening shot scattering the racked balls) identically to mid-game shots, missing opportunities for specialized break strategies that could improve ball group assignment.

Multi-Step Planning: Current decisions optimize single shots without explicit planning for subsequent shots. While strategic bonuses provide indirect multi-step reasoning through positional rewards, a lookahead search (e.g., 2–3 shots) could improve end-game play.

Opponent Modeling: The agent does not adapt to opponent behavior (e.g., exploiting predictable tendencies). Online opponent modeling could yield marginal win rate gains in extended matches.

Offline Training: We operate in a zero-shot manner without offline learning. Pre-training a value function or policy network could replace hand-crafted reward components with learned representations, potentially improving generalization.

B. Future Directions

Several promising extensions warrant exploration:

Hybrid Neural-Geometric Approaches: Combine learned value networks (estimating game-theoretic value of resulting states) with geometric shot generation. This could improve defensive play and long-term planning while retaining computational efficiency.

Meta-Learning for Adaptive Pruning: Learn pruning heuristics from historical games to dynamically adjust candidate generation based on table configuration. For instance, increase candidates in complex cluttered states, decrease in simple sparse states.

Parallelized Physics Simulation: Exploit GPU acceleration or distributed computing to evaluate candidates in parallel, potentially allowing more thorough CMA-ES optimization without exceeding time budgets.

Transfer to Real-World Robotics: Adapt the framework to physical billiards robots, addressing additional challenges like visual perception, mechanical actuation noise, and real-time control loop delays.

Generalization to Other Domains: The geometric pruning + selective evolution strategy may transfer to other continuous control domains with structured constraints (e.g., robotic manipulation, drone navigation, industrial process control).

In summary, this work demonstrates that intelligent integration of domain knowledge with general-purpose optimization can yield favorable trade-offs in time-constrained decision-making. The lessons learned—prioritize candidate quality over quantity, adapt computational budget to difficulty, and filter catastrophic failures early—apply broadly to AI systems operating under resource constraints.

PROJECT CONTRIBUTION AND DIVISION OF LABOR

Team Statement: This project was completed by a two-person team (Jifan Lin and Yifan Yang). The manuscript is submitted as an individual course report, with the division of labor and contributions summarized below.

Jifan Lin (523030910057):

- *Algorithm design and implementation:* Designed and implemented the hybrid Ghost Ball + CMA-ES decision pipeline, including geometric pruning, layered stochastic evaluation, catastrophic foul filtering, and safety/positional shot planning.
- *Engineering optimization:* Iteratively profiled runtime bottlenecks and redesigned the pipeline to satisfy time constraints while maintaining competitive win rate.
- *Manuscript preparation:* Wrote and typeset this IEEE-format report, including formalization, pseudocode, and experimental discussion.

Yifan Yang (523030910054):

- *Evaluation infrastructure and reproducibility:* Prepared and maintained the evaluation environment, validated baseline agents, and ensured tournament-style testing is reproducible across machines and seeds.
- *Ablation study leadership:* Led the design and execution of ablation experiments (Table III), including selecting ablation variants, running large-N match suites, and cross-checking summaries/plots for consistency.
- *Hyperparameter tuning and robustness:* Performed targeted tuning and stress testing (candidate caps, sampling schedules, pruning thresholds, and strategy weights), contributing to improved win rate and runtime stability under stochastic action noise.
- *Review and quality control:* Systematically reviewed results, figures, and manuscript consistency (numbers, units, and captions), and provided feedback that improved clarity and compliance with course requirements.

ACKNOWLEDGMENT

The authors thank the AI3603 course staff at Shanghai Jiao Tong University for providing the Pooltool simulation environment and baseline agents. Special thanks to the developers of the Pooltool library for maintaining an open-source, high-fidelity billiards physics engine. This project was completed collaboratively by a two-person team; this manuscript is submitted as an individual course report with the division of labor stated above.

REFERENCES

[1] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

[2] E. Brochu, V. M. Cora, and N. de Freitas, "A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning," *arXiv preprint arXiv:1012.2599*, 2010. [Online]. Available: https://arxiv.org/abs/1012.2599

[3] N. Hansen, S. D. M"uller, and P. Koumoutsakos, "Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES)," *Evolutionary Computation*, vol. 11, no. 1, pp. 1–18, 2003.

[4] D. Pérez-Liébana, S. Samothrakis, S. M. Lucas, and P. Rohlfshagen, "Rolling horizon evolution versus tree search for navigation in single-player real-time games," in *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation (GECCO '13)*. ACM, 2013, pp. 351–358.

[5] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *Computers and Games*, ser. Lecture Notes in Computer Science, H. J. van den Herik, P. Ciancarini, and H. H. L. M. Donkers, Eds., vol. 4630. Springer, 2007, pp. 72–83.

[6] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *Machine Learning: ECML 2006*, ser. Lecture Notes in Computer Science, J. F"urnkranz, T. Scheffer, and M. Spiliopoulou, Eds., vol. 4212. Berlin, Heidelberg: Springer, 2006, pp. 282–293.

[7] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[8] SJTU-RL2, "pooltool (fork of ekiefl/pooltool)," https://github.com/SJTU-RL2/pooltool, gitHub repository. Accessed: 2026-01-04.

[9] CueSports International, "Official rules of cuesports international (used by the bcapl and the usapl)," https://www.playcsipool.com/uploads/7/3/5/9/7359673/official_rules_of_csi__08122025.pdf, 2023, effective June 1, 2023. Accessed: 2026-01-04.